

- Project 2: Report
 - Intro (Your understanding)
 - Flow of Control & Code Design
 - 1. The ViewServer Flow
 - 2. The PBServer (Replica) Flow
 - 3. The Client Flow
 - Design Decisions
 - Missing Components
 - References
 - Extra (Optional)

Project 2: Report

Intro (Your understanding)

This lab implements a fault-tolerant, distributed Key-Value store using a Primary-Backup replication protocol. To ensure strong consistency and prevent "split-brain" scenarios, the system relies on a central, unreplicated master node called the **ViewServer**.

The **ViewServer** acts as an oracle for the system's current configuration (the "View"). It monitors the liveness of all registered servers via periodic heartbeats. If the Primary fails, it promotes the Backup. If the Backup fails, it recruits an idle server. The **PBServer** nodes handle the actual client workloads, forwarding state updates from the Primary to the Backup to maintain identical replicas. The system guarantees linearizability and exactly-once execution (via the **AMOApplication** shim) even in the presence of network partitions, message reordering, and node crashes.

Flow of Control & Code Design

1. The ViewServer Flow

- **Liveness Tracking:** Maintains a **missesRemaining** counter for each server. The **PingCheckTimer** decrements this counter for all servers periodically. If a **Ping** is received, the counter is reset, granting the server a new "lease" on life.

- **View Generation:** When a Primary or Backup dies (counter reaches 0), or a new server joins, the **ViewServer** attempts to generate a new View based on the available and alive servers based on the following rules:
 - If a primary dies, only the backup of the current view can become the primary in the next view to ensure data integrity.
 - Any available, alive server can become the Backup in the next view.
- **The Acknowledgement Rule:** The **ViewServer** will strictly **not** increment the view number or change roles unless the Primary of the current view has explicitly acknowledged it via a **Ping(currentViewNum)**.

2. The PBServer (Replica) Flow

- **Initialization (State Transfer):**
 - When a server is promoted to Primary with a new Backup, it halts regular command processing. It takes a complete snapshot of its **AMOApplication** (which includes both the KVStore data and the deduplication history) and sends it to the Backup via a **PBInitRequest**.
 - It sets a **InitTimer** to keep retrying until it receives a successful state initialization response from the server, **PBInitReply**.
 - On receiving **PBInitReply**, it does the following:
 - Sends an acknowledgement to the **ViewServer** confirming the state initialization is complete and the view is active.
 - Initiates a **PBCommandTimer** to resume processing and send queued client commands to the backup.
- **Normal Operation:**
 1. The Primary receives a **CSRequest** from a client.
 2. It executes the command locally on its **AMOApplication**.
 3. If it is a Read-Only command (e.g., **Get**), it replies to the client immediately.
 4. If it is a Write command (e.g., **Append**, **Put**), it queues the command and forwards it to the Backup via a **PBCommandRequest**.
 5. On receiving a **PBCommandTimer**, the Primary retries sending the command at the front of the pending queue to the Backup to handle potential packet loss.
 6. Once the Backup processes the command and replies with a **PBCommandReply**, the Primary removes the command from the queue and finally sends the **CSReply** back to the Client.

- **Heartbeats:** Servers continuously fire a **PingTimer** to send their current view number to the **ViewServer**.

3. The Client Flow

- The client uses a stop-and-wait approach, setting a **ClientTimer** upon sending a request. If a request times out, it assumes network failure or server crash and resends the request to the Primary.
- If the server replies with an error/view-mismatch, the client sends a **GetView** request to the **ViewServer** to discover the new Primary and retries the command.
- The client also sends **GetView** messages every **PING_MILLIS** to the **ViewServer** to get the latest view of the distributed system.

Design Decisions

Our final implementation was heavily influenced by the aggressive network permutations introduced by the search tests.

- **Sending the full AMOApplication for State Transfer:** Initially, I tried to synchronize the Backup by extracting the KVStore state. However, if a Primary crashes, the new Primary must know which client sequence numbers have already been executed to preserve "Exactly-Once" semantics.
 - *Decision:* I designed the **PBInitRequest** to serialize and transmit the entire **AMOApplication** wrapper. This instantly synchronizes both the database state and the deduplication history, preventing old, delayed client requests from being applied twice on the newly promoted Primary.
- **Idempotent Backup Initialization (The Delayed Duplicate Trap):** Search tests revealed that a delayed **PBInitReply** causes the Primary to re-send the **PBInitRequest**. If the Backup blindly accepted this duplicate *after* it had processed new client commands, it would overwrite its state with the old snapshot, erasing history.
 - *Decision:* I introduced a **backupStateInitialized** flag. The Backup only accepts the **AMOApplication** payload once per view. Subsequent duplicate init requests are ignored, though an acknowledgement is still sent to satisfy the Primary's timer.
- **Strict View Acknowledgements to Prevent Split-Brain:** If a Primary immediately acknowledges a new View to the **ViewServer** *before* the Backup is

fully initialized, a subsequent Primary crash will cause the `ViewServer` to promote an empty Backup, leading to permanent data loss.

- *Decision:* The Primary suppresses its acknowledgement (sending `Ping(viewNum - 1)`) until it receives a successful `PBInitReply` from the Backup.

Missing Components

None. The system successfully passes all run tests and liveness/correctness search tests, effectively handling message reordering, partitions, and node crashes.

References

- MIT 6.824 Lab 2 / DSLabs Handout documentation.
- DSLabs Visual Debugger (Crucial for visualizing counter-examples and trace isolation during Search Test timeouts).
- Java Documentation for object serialization and `HashMap` concurrency behaviors.

Extra (Optional)